StoreOS — Antigravity Git Workflow & Full Build Task List

For Abrar — handoff document for Google Antigravity to (1) auto-commit every change to GitHub, and (2) execute the remaining build phases independently, phase by phase, with explicit test gates.

Part 1 — Git Auto-Commit Workflow

Before you start: one-time setup (do this yourself, not Antigravity)

1. Create a GitHub repo (e.g. `storeos`), private if the codebase will ever contain anything sensitive.
2. In the project root, if git isn't already initialized:

```
git init
git remote add origin https://github.com/<your-username>/storeos.git
```

3. Confirm `.env*` stays in `.gitignore` (it already is, per your current `.gitignore`) — Antigravity must never commit secrets.
4. Push an initial commit yourself so `origin/main` exists before handing off to Antigravity.

The prompt — paste this into Antigravity as a standing instruction

```
GIT WORKFLOW — MANDATORY, EVERY SESSION

From now on, treat git commits as part of "done," not an afterthought.

1. Before starting any task, run `git status` and `git pull --rebase origin
main`
   to make sure you're working from the latest state.

2. Work in small, reviewable increments. After EACH meaningful change
   (a completed feature, a fixed bug, a finished sub-task from the work list
—
   not every single file edit), do the following before moving to the next
task:

   a. Run the project's type check and any available tests. Do not commit
      code that fails type checks.
   b. Stage only the files relevant to that change: `git add <files>`
```

```
      (avoid `git add .` if it would sweep in unrelated scratch files).
   c. Commit with a clear, conventional message:
      - feat(pos): add cart discount calculation
      - fix(sync): resolve duplicate invoice on retry
      - chore(schema): add stock_movements index
      - test(sync): add idempotency test for retried writes
   d. Push to the remote: `git push origin main`
      (or to a feature branch if I've asked you to use one).

3. NEVER commit:
   - .env files or any file containing API keys/secrets
   - node_modules, .next, build artifacts
   - commented-out dead code "just in case"

4. If a change is large or risky (schema migration, auth flow, sync engine
   changes), commit it separately from smaller changes so it can be reverted
   in isolation if something breaks.

5. At the end of every work session, give me a short summary of what was
   committed (commit hashes + one-line description each), what's still
   uncommitted/WIP, and what failed type checks or tests, if anything.

6. If you are ever unsure whether something should be committed yet
   (e.g. a half-finished feature), ask me rather than committing broken
   code to main. Prefer a WIP branch over a broken main.
```

Part 2 — Full Work List (Phase-by-Phase, for Antigravity to execute independently)

Preface note for Antigravity: Phase 1 was already reported complete, but the status report did not confirm several of the "critical risk" items from the original spec. Before starting Phase 2+, re-verify Phase 1's foundations below — do not assume they're correct just because the build compiles.

Phase 0 — Verification pass on what's already built (do this first)

- [ ] Confirm `client_generated_id` is enforced as a unique/idempotency constraint on every syncable table server-side (not just client-side).
- [ ] Confirm RLS policies exist and are tested for every table in the spec (not just that `business_id` columns exist — that policies actually restrict access).
- [ ] Write and run a test: simulate a retried sync of the same invoice (same `client_generated_id`) and confirm no duplicate row is created.

- [ ] Write and run a test: simulate two devices offline, both logging stock movements against the same product, both syncing — confirm the final quantity is correct via delta replay, not overwritten.
- [ ] Confirm money-table conflicts (sales, payments) are surfaced in a "Sync Issues" UI for manual review, and do NOT silently auto-merge. Write a test that forces a conflict and asserts it lands in the conflict queue, not applied automatically.
- [ ] Confirm offline invoice numbering is collision-free across two simulated offline devices syncing the same batch window.

Report back pass/fail on each item before proceeding. Fix any failures before Phase 2.

Phase 2 — Product & Inventory Core (verify/extend)

- [ ] Categories CRUD (nested via `parent_id`)
- [ ] Products CRUD with Zod validation on client and server
- [ ] Product variants (`attribute_json`-based, e.g. size/color)
- [ ] Barcode/SKU lookup, including scanner input handling in POS search
- [ ] Batch tracking (expiry-date-aware) for products with `has_batch_tracking`
- [ ] Serial tracking for products with `has_serial_tracking`
- [ ] Low-stock threshold logic + alerting on the dashboard
- [ ] All writes go through local Dexie first, enqueue to syncQueue, return immediately

Test gate: full CRUD cycle for a product with variants, offline, then sync, with data integrity confirmed on both local and remote after sync.

Phase 3 — POS + Sales

- [ ] Cart (add/remove/adjust qty), real-time subtotal/discount/tax calc using integer minor-unit math only (no floats anywhere in money logic — review for stray `/100` or float literals)
- [ ] Multiple payment methods, split/partial payments
- [ ] Automatic due-balance update on partial payment
- [ ] Offline invoice numbering: branch/device-prefixed, reconciled to canonical sequence on sync — confirm the reconciliation step actually exists, not just prefix generation
- [ ] Receipt generation (print + PDF)
- [ ] Returns hook point (so Phase 6 can plug in without rework)

Test gate: full sale (cart → payment → invoice → stock movement) end-to-end offline, then sync, with stock and dues correctly reflected server-side.

Phase 4 — Customers & Due Collection

- [ ] Customer CRUD, opening due balance
- [ ] Due ledger per customer, searchable by name/phone
- [ ] Payment collection against due balance
- [ ] Customer statement (printable/exportable history of invoices + payments)

Test gate: due balance stays correct across an offline sale → offline partial payment → sync sequence (no double-counting, no lost payments).

Phase 5 — Suppliers & Purchasing

- [ ] Supplier CRUD
- [ ] Purchase orders → purchase items
- [ ] Receiving stock generates proper stock_movements (type: purchase), not direct quantity edits

Test gate: receiving a purchase order correctly increments stock via the movement log, syncs cleanly, and is reflected in Phase 2's low-stock alerts.

Phase 6 — Returns & Exchanges

- [ ] Sales returns (full/partial), linked to original invoice
- [ ] Refund handling across all payment methods
- [ ] Stock reversal movements (additive delta, consistent with the Phase 2/3 model)

Test gate: a partial return correctly reverses stock and adjusts customer due/refund without breaking the original invoice's audit trail.

Phase 7 — Expenses & Employees

- [ ] Expense CRUD with categories
- [ ] Employee CRUD, salary tracking
- [ ] Hook expenses into the dashboard's "Net Profit" calc (sales − cost − expenses)

Test gate: dashboard net profit figure changes correctly when an expense is added offline and synced.

Phase 8 — Dashboard & Reports (review/extend what exists)

- [ ] Confirm KPI cards pull from real Dexie data, not mocked values (the earlier status report's sample numbers — confirm they're live, not hardcoded placeholders)
- [ ] CSV export correctness: spot-check exported totals against the UI
- [ ] Stock valuation report

Test gate: reports match the underlying ledger exactly for a test dataset constructed and checked by hand.

Phase 9 — Multi-Branch

- [ ] Branch-scoped data isolation, enforced at RLS level (not just UI filtering)
- [ ] Cross-branch permission boundaries (a branch staff account cannot see another branch's data — confirm via direct Supabase query test, not just UI)
- [ ] Branch switcher for business owners with multi-branch access

Test gate: log in as a branch-scoped role and attempt to query another branch's data directly against Supabase — confirm RLS blocks it.

Phase 10 — Hardening

- [ ] Full Sync Issues UI: list of conflicts/failures with manual resolution actions
- [ ] Backup/restore: export + re-import a full IndexedDB snapshot, confirm round-trip integrity
- [ ] Audit log UI: browsable, filterable by entity/user/date
- [ ] Permission management UI (tied to `roles_permissions`)
- [ ] Performance pass: test with a large catalog (5,000+ products) for POS search lag and IndexedDB query times
- [ ] Full test suite pass: unit (money math, validation), integration (full sale flow), sync tests (idempotency, conflict, retry/backoff), E2E for POS/due/returns/offline-online transition

Test gate: this phase doesn't end until every item in the spec's original "Critical Risks" table has a corresponding passing test, not just a UI that looks right.

How Antigravity should report progress

After each phase, require a short report in this format:

```
Phase N — [name]: COMPLETE / PARTIAL / BLOCKED
- What was built:
- What was tested (and result):
- What's stubbed or deferred:
- Commits: [hash] message, [hash] message...
- Known issues / things needing your review:
```